

```
LOG:  redo done at 0/2000138 system usage: CPU: user: 0.00 s, system: 0.00
s, elapsed: 0.00 s
LOG:  database system is ready to accept connections
done
server started
```

Here, the server has been started on the cloned PGDATA directory and TCP/IP port 5433. If you inspect the database cluster logs, you will notice that PostgreSQL claims to have been interrupted and that a redo process started, and then completed. Once the redo is completed, the database is ready to start its normal activity. In short, this is a crash-like situation: the physical dump has copied a “dirty” situation of the database, but thanks to the WALs, it is able to self-heal.

You will see that the database has been restored from a “forced crash,” that is, the cloned cluster did self-healing on its first startup.

Restoring from a physical backup

If you need to restore from a physical backup, you need to overwrite the original PGDATA directory with the cloned copy produced by `pg_basebackup`. This is a very risky operation because you will lose all the content of the PGDATA directory and replace it with the backup copy, which means the risk of errors occurring is high.

For that reason, instead of performing an online restoration, we suggest that you start a cloned cluster somewhere else, as shown in the previous section, so that you can extract the data you need to recover and restore only that data on the target cluster. For instance, you can start the cloned server, extract the data you need to recover by means of `pg_dump`, and restore it on the target cluster.

Of course, there are situations when you need to recover the entirety of the cluster, and therefore, you need to do PGDATA overwriting, but even in such cases, we suggest that you use more advanced tools such as `pgBackRest` that drive and assist you in both the backup and restore part.

Physical backup and restoration are very powerful mechanisms, but they require you to deeply understand what is going on under the hood. So, take the time to experiment with them carefully so that you’re ready to apply them in production.

Basic concepts behind PITR

Point in Time Recovery, usually written as PITR, is a technique that allows you to restore your database at a specific point in the past. Showing you how to use PITR is out of the scope of this chapter, and this section only explains the basic concepts behind the technique.

PITR can be achieved only by means of physical backup, and it is usually performed via specific backup tools like the aforementioned pgBackRest, even though PostgreSQL provides all the needed infrastructure to perform PITR.

The main idea behind PITR is to start with a physical backup and then continuously store the database WAL segments, a process called *WAL archiving*. The WALs can be stored locally or sent to a remote machine, usually a specific backup machine. The need to archive all the WALs is that, as already explained in *Chapter 11*, PostgreSQL recycles the WALs once the modified data is safely stored on the disk; therefore, in order to get a continuous stream of WALs, the administrator needs to keep all of them. PostgreSQL provides a specific configuration setting, named `archive_command`, that can be tuned to execute an external command (e.g., copy commands like `cp`, `scp`, `sftp`, and so on); `archive_command` is executed on every single WAL segment as soon as PostgreSQL has completed the WAL file and switches to a new segment.

Having the physical backup and the stream of WALs, the database cluster can be instructed to replay all the transactions (contained in the WALs) one after the other until the expected restore time is reached. At such a point, the cluster could ignore exceeding WALs and start from there as a separate and new instance.

Therefore, the idea behind PITR is to start from a physical backup and let the cluster move forward in time until it has reached the desired instance. The point of restoration can be specified as a timestamp, thus indicating a specific time instance, or a transaction identifier, or it can even be a *label* that the database administrator has placed into the WALs, without them having to worry about either the time or transactions occurring.

One disadvantage of PITR is that it requires some time and effort for the cluster to be restored to a given time. Moreover, if a single WAL segment is lacking in the stream, the cluster will not be able to recover at all. For that reason, using dedicated backup tools is usually the best option to manage PITR.

Summary

In this chapter, we learned that PostgreSQL provides advanced tools so that we can perform backups and restorations. Backups are important because, even in a battle-tested and high-quality product such as PostgreSQL, things can go wrong: often, users may accidentally damage their data, but other times, the hardware or the software could fail miserably. Being able to restore data, partially or fully, is, therefore, very important, and every database administrator should carefully plan backup strategies.

We also learned that PostgreSQL ships with tools for both logical and physical backups. Logical backups are taken by means of reading the data from the database itself, using ordinary SQL interactions; physical backups are taken by means of cloning the PGDATA directory, either by using operating system tools or PostgreSQL ad hoc solutions. Restoration is performed by specific tools in the case of logical backups, and by the database self-healing mechanism in the case of physical backups.

Finally, it is important to stress the concept that a backup alone is not valid until it is successfully restored, so to ensure that you will be able to recover your cluster, you need to test your backups as well.

Now that you can back up and restore your clusters, in the next chapter, we will look at configuration and monitoring.

Verify your knowledge

- What is the difference between a logical and physical backup?

A logical backup, also known as a “dump,” is a backup that interacts directly with the database and its running transactions. A physical backup, also known as a “hot backup,” is a copy of the underlying filesystem and the WALs so that the last known clear state of the database can be restored. See the *Introducing types of backups and restores* section for more detail.

- What is the difference between `pg_dump` and `pg_dumpall`?

The `pg_dump` command is used to dump a single database, while `pg_dumpall` dumps all the intra-database objects (e.g., the users) and then performs a `pg_dump` against every database in the cluster. See the *Dumping a whole cluster* section for more details.

- What is the COPY command?

The COPY command is a PostgreSQL-specific statement aimed at bulk-loading or extracting data. It is often used as a way to dump/restore data from a table. See the *The COPY command* section for more details.

- What is the pg_basebackup command?

The pg_basebackup command is a command that performs a physical backup of a running cluster and can also archive WALs. See the *Performing a manual physical backup* section for more details.

- What is Point in Time Recovery (PITR)?

PITR is a technique by which a physical backup is restored to a specific point in time after the backup has started. It is a way to take a cluster *back in time*. See the *Basic concepts behind PITR* section for more details.

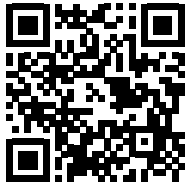
References

- PostgreSQL pg_dump tool official documentation: <https://www.postgresql.org/docs/current/app-pgdump.html>
- PostgreSQL pg_dumpall tool official documentation: <https://www.postgresql.org/docs/current/app-pg-dumpall.html>
- PostgreSQL pg_restore tool official documentation: <https://www.postgresql.org/docs/current/app-pgrestore.html>
- FreeBSD 502.pgsql backup script: <https://www.freshports.org/databases/postgresql83-server/files/502.pgsql>
- PostgreSQL pg_basebackup tool official documentation: <https://www.postgresql.org/docs/current/app-pgbasebackup.html>
- PostgreSQL pg_verifybackup tool official documentation: <https://www.postgresql.org/docs/current/app-pgverifybackup.html>
- PostgreSQL COPY command official documentation: <https://www.postgresql.org/docs/current/sql-copy.html>
- pgBackRest external tool for physical backups: <https://pgbackrest.org/>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



16

Configuration and Monitoring

One of the duties of a database administrator is to configure the cluster so that it behaves well for the current workload and context. The configuration is not static: most of the time, you will find yourself making changes to the configuration, so it is important that you feel comfortable with inspecting and changing the cluster's configuration.

Another important task, partially related to configuration, is monitoring the cluster in order to understand how the system is actually behaving and whether there are bottlenecks and problems to be solved. Such problems can sometimes be solved by making changes to the configuration of the cluster, by using different hardware (for example, increasing the available memory), and by fixing the applications that could be causing the bottleneck.

This chapter will show you how to manage and inspect the cluster configuration, generate a configuration from scratch, find errors and mistakes in the current configuration, and interactively monitor the cluster's activity via the rich statistics subsystem. Finally, you will discover a very powerful and commonly used extension, named `pg_stat_statements`, that allows you to monitor the cluster's activity with great detail and flexibility.

This chapter will cover the following topics:

- Cluster configuration
- Monitoring the cluster
- Advanced statistics with `pg_stat_statements`

Let's get started!

Technical requirements

You need to know about the following to complete this chapter:

- How to interact with configuration files within the PGDATA directory
- How to connect to your cluster as a database administrator
- How to execute SQL statements against the system catalogs

The chapter examples can be run on the `chapter_16` Docker image, which you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>.

Cluster configuration

PostgreSQL is configured by means of a bunch of text files that contain directives and values used to bootstrap the cluster and get it running. We have seen how configuration files are handled throughout the book, but in this chapter, we will go into detail explaining how configuration is managed.

There are three main configuration files that present the *starting point* for any configuration:

- `postgresql.conf` is the main cluster configuration file and contains all the data required to start the cluster, set up processes (as WAL senders) and logging, and configure how the cluster will accept connections (for example, on which TCP/IP address).
- `postgresql.auto.conf` is a file automatically generated and edited by the cluster itself and contains parameters changed by the superuser from within the cluster. You should never edit this file manually, but you can inspect it with your text editor to read its contents.
- `pg_hba.conf` is the file that's used to allow or deny the client connections to the cluster. It was explained extensively in *Chapter 3, Managing Users and Connections*, and is related to the users and roles authentication mechanisms.

Usually, both of the above files are within the PGDATA directory where all the cluster data lives, but a few operating systems may place such files into other directories; as an example, usually, Debian GNU Linux places them into `/etc/postgresql`.



In the Docker images of the book, the configuration files are kept under the PGDATA directory, i.e., in `/postgres/16/data`.

There are other configuration files usually kept within the PGDATA directory, but they will not be discussed here. This section will mainly be dedicated to `postgresql.conf`, the default configuration file.

The `postgresql.conf` file is a text file, usually annotated with useful comments, that contains a set of configuration parameters. Each parameter is expressed in the form of `key = value`, where the **key** is the configuration parameter name and the **value** is the configuration value for the parameter. We saw a few configuration parameters in the previous chapters. For example, the `max_wal_senders = 2` configuration parameter sets the `max_wal_senders` configuration parameter to the value of 2.

Each configuration parameter must be on a single line, and all lines starting with a `#` sign are comments, which will not be taken into account by the cluster. Comments are useful since they allow you to add extra information about your intentions regarding a specific configuration. For example, let's take a look at the following code snippet:

```
# set to 2 to allow pg_basebackup to work properly
max_wal_senders = 2
```

The previous example provides a clear hint about why the parameter has been configured as such. You are not required to place comments in your configuration file, but it is a very good habit to document what you are doing and why you are doing it.

A parameter can be defined multiple times; in such a case, the last definition found is the one that PostgreSQL uses.

If you don't want to configure a specific parameter, you can either delete the line of that parameter, making it disappear totally from the configuration file, or place a comment sign in front of it, transforming the line into a pure comment. It is important to note that if a parameter is not configured in the file because it is either missing or commented out, then it will take its default value. Every parameter has a default value, and you must look through the documentation to understand each default value; however, a default installation configuration file contains all the available parameters with comments that explain their usage and default settings.

Each configuration parameter will accept only a specific set of values that depend on the type of the configuration parameter itself. Mainly, you can encounter numeric values, string values, and lists (separated by a comma); there are also values that can be expressed with a measurement unit, such as times, like `2ms` (2 milliseconds), or sizes, like `2GB` (2 gigabytes).

Inspecting all the configuration parameters

You can inspect all the configuration parameters from a live system by issuing a query against the `pg_settings` special catalog. This catalog contains every setting that the current version of PostgreSQL will accept, along with default values, current values, a description of the parameter aim, and much more.

As an example, with the following query, you can gather information about every configuration parameter, including their short and long descriptions, default values, and current values:

```

forumdb=> SELECT name, setting || ' ' || unit AS current_value, short_
desc, extra_desc, min_val, max_val, boot_val, reset_val FROM pg_settings;
...
name          | authentication_timeout
current_value | 360 s
short_desc    | Sets the maximum allowed time to complete client
authentication.
extra_desc    |
min_val       | 1
max_val       | 600
boot_val      | 60
reset_val     | 360

```

In the preceding example, the `authentication_timeout` has been set to 360 seconds, as indicated by the `setting` column, and its value can be tuned in the range of 1 second to 600 seconds; `boot_val` is the default value that the parameter will assume if it is not configured at all, and it is set to 60 seconds. Similarly, the `reset_val` column indicates the value the parameter will assume if it is reset after a change.

The `pg_settings` special catalog also contains other useful information, including the file and the line number from where a parameter has been loaded. This information can be used to quickly find where a configuration parameter has been set in the `postgresql.conf` file or another configuration file. As an example, the following query will show where each parameter has been loaded from:

```

forumdb=# SELECT name, setting AS current_value, sourcefile, sourceline,
pending_restart FROM pg_settings;
...
name          | log_destination

```

```

current_value | stderr
sourcefile    | /postgres/16/data/postgresql.auto.conf
sourceline    | 4
pending_restart | f

```

As you can see, in this example installation, the `log_destination` configuration parameter has been loaded from the `/postgres/16/data/postgresql.auto.conf` file. This is not the main `postgresql.conf` file; it is a file that PostgreSQL automatically generates to handle configuration parameters changed from the inside of the cluster. The usage of `postgresql.auto.conf` will become clear later on in this chapter; for now, it suffices to know that a configuration parameter has been loaded from another file and that `pg_settings` clearly reports that.



You need to have database superuser rights in order to gather extra information, such as the file location and line number.

There is another important piece of information that you can get out of the `pg_settings` special catalog: what happens if a parameter is changed at runtime? Depending on the nature of the parameter, changes can be applied immediately, or they can be delayed while waiting for a special event or even a cluster restart. The `pending_restart` column indicates whether the current parameter has changed from its boot time value and whether the value has been applied to the cluster. In the previous example, `pending_restart` is false, so the configuration you are seeing is effectively what is running on the cluster right now.

Querying `pg_settings` provides you with all the information you need to understand the status of the current configuration, but there is a shortcut to get the setting of a parameter in a quicker way: the `SHOW` command. The special command `SHOW` accepts the name of a parameter and reports back, in a human-understandable form, the value of such a parameter. As an example, imagine we want to inspect the `shared_buffers` memory value:

```

forumbd=> SHOW shared_buffers ;
shared_buffers
-----
128MB
(1 row)

```